

A Deterministic Alignment Engine for Meaning

The Recursive Decompression Protocol (RDP) and Structural Ontology Alignment Method (SOAM) for the Symbol Grounding Framework

SGF Subsystem Specification v3.2 — June 2026

Author: James Lee Stäkelum

Abstract

We present the **Recursive Decompression Protocol (RDP)** — a lexicon construction and deterministic alignment subsystem for the Symbol Grounding Framework (SGF). The RDP compiles natural language into structured, grounded concept definitions using SGF's orthogonal axes architecture (Y-axis object logic with `IS_A` polyhierarchies and `HAS_PART` composition; X-axis event logic with VerbHub and 15 fixed semantic roles), executes deterministic alignment via the **Structural Ontology Alignment Method (SOAM)** using recursive bisatisfiability checking with nested logical operators and metric-aware constraint matching, and stores all state in a persistent adjacency manifest.

The system is presented as a derivational chain. Each component exists because a prior solution created a new problem that only that component can solve. The fingerprint pre-filter exists because full decomposition on every query is too slow; the three-level depth hierarchy exists because different decisions have different consequences; the ProofTrace exists because procurement officers must be able to verify, not just guess.

The architecture yields four capabilities that span from a single concept to a full procurement specification: privacy-preserving verification via an

86-character fingerprint, cross-lingual interoperability without runtime translation, a unified call center engine that eliminates the trillion-dollar guessing cycle, and RFP-to-proposal response verification with item-by-item ProofTraces. A fifth capability — the verifiable interlingua — emerges from combining SOAM vocabulary alignment with the Synapse grammar's language-neutral proposition format.

Language is vocabulary plus grammar. SOAM provides the vocabulary layer — deterministic concept alignment with ProofTraces. The Synapse format provides the grammar layer — a fixed-role proposition structure that is itself alignable by the same engine. Together they form a complete, verifiable interlingua: the vocabulary tells you what things mean; the grammar tells you who did what to whom. Both are auditable, both are consequence-sensitive, and both use the same deterministic architecture.

The RDP fills a specific gap within SGF: SGF provides the architecture for representing and transporting grounded meaning, but did not previously specify a deterministic method for comparing two concept definitions to determine equivalence. This paper provides that method. A natural extension — applying SOAM's recursive bisatisfiability to SGF's Synapse format for grammar alignment across languages — is identified as future work.

Table of Contents

1. Introduction
2. The Problem: Lexicon Alignment in High-Consequence Domains
3. Related Work
4. Design Principles and Axioms
5. The Schema: A Problem-Derived Framework for Concept Definition
6. The Rules of the Map
7. Quality Tests
8. The Receiver Policy: Consequence Forces Admission Standards

9. Bootstrapping and Semantic Alignment
10. The Structural Ontology Alignment Engine (SOAM)
11. Use Cases
12. The Semantic Firewall Router
13. Limitations
14. Conclusion and Future Work
References
Appendix A: Slot Name Mapping
Appendix B: NSM Prime Registry
Appendix C: Exact Profile Contract Specification
Appendix D: Terminology

1. Introduction

The Symbol Grounding Framework (SGF) is an architecture for representing, transporting, and governing grounded machine meaning. It provides a Core Lexicon with Canonical IDs and Prime Registry bedrock, a Synapse grammar with verb-at-hub and 15 fixed semantic roles, GLEAN for text ingestion, HFF for transport, AFP for machine acts, and Omega for governance.

The RDP fills a specific gap within SGF: SGF describes what grounded meaning looks like and how to transport it, but it does not specify a deterministic method for comparing two independently-built concept definitions to determine whether they refer to equivalent things. The RDP provides this method. It solves **semantic alignment** — making sure two systems mean the same thing by the same symbols. This is distinct from, and a prerequisite for, the broader problems of goal specification (writing down the right objective) and value learning (inferring what humans actually care about). The paper addresses only semantic alignment; it does not claim to solve the full AI alignment challenge.

Because the grammar is fixed (the 15 semantic roles, `IS_A`, `HAS_PART`) and the vocabulary is open with tethers back to the Core Lexicon, two systems can align concepts at runtime without prior integration. Integration becomes a discovery event, not a design-time prerequisite. This is the architectural mechanism that dismantles the Babel Tax of legacy ontology alignment.

One-shot scope. This paper is written as a single, self-contained specification. It describes the full architecture — from concept alignment to requirement verification — because most readers will encounter only this document. The three layers (vocabulary, grammar, requirements) share the same engine, the same ProofTrace format, and the same consequence-sensitive depth policy. The reader who finishes this paper will understand how SOAM scales from a single screwdriver to a 25-page procurement specification.

2. The Problem: Lexicon Alignment in High-Consequence Domains

2.1 The Core Failure

Federated procurement, industrial coordination, and multi-enterprise knowledge sharing across sovereign defense networks are paralyzed by a single, recurring failure: **two high-fidelity technical lexicons cannot efficiently determine whether they refer to the same grounded concepts.**

2.2 The Cost of Misalignment

A customer calls a parts hotline and describes a brake rotor for a 1997 Toyota Tacoma. The search system returns three candidates. The customer guesses. The wrong part ships.

This cycle repeats millions of times a day across every industry with a high-SKU catalog. Conservative estimates place the annual cost of

misordered parts, return logistics, and lost customer trust at **over a trillion dollars** globally.

But the consequences scale.

Consider a procurement officer at a defense agency evaluating a vendor's proposal for a "flight-qualified torque driver." The specification references ISO 10241 for fastener terminology, MIL-STD-171 for finishing requirements, and NATO STANAG 4107 for mutual recognition of component inspections. The vendor's proposal uses internal part numbers, proprietary material grades, and vague clauses like "exceeds required strength margins."

The officer must determine if the vendor's torque driver is the same as the spec's torque driver.

A false positive — admitting a non-conforming part — can be catastrophic. But the most vivid illustration comes from a story that inspired this system:

The NASA Screwdriver Problem. A procurement officer ordered a torque driver for a NASA mission. The spec called for "extreme environment rated — vacuum, zero gravity, arctic, underwater." The system returned three candidates. The officer picked one. The part that shipped was rated for atmospheric use only. It made it onto the space station. When the astronauts tried to use it, it failed. The cost to send the correct screwdriver up on the next resupply: **a billion dollars**.

This is the NASA screwdriver problem — the moment when a high-stakes product match is left to luck instead of proof.

Current systems cannot solve this because they lack the ability to **verify** — they can only guess, rank, and recommend.

2.3 Why Current Approaches Fail — And What This Forces

Keyword matching cannot distinguish a "socket head cap screw" from a "set screw" — both share keywords like "screw" and "head." Pure embedding similarity cannot enforce that a "screwdriver" must be a "tool," not a "cocktail

ingredient" — the embedding for "screwdriver" as a tool and "screwdriver" as a drink may be close. RDF triple stores treat every edge label as a unique predicate, producing unbounded vocabulary growth and N-squared integration costs. Human-in-the-loop review does not scale.

This forces a different approach. The approach must be deterministic — it must produce the same answer every time for the same inputs. It must be verifiable — the procurement officer must be able to see why a match was accepted or rejected. And it must be grounded — every term must trace back to a shared bedrock that neither party can redefine.

2.4 What Determinism Requires

To decide "does concept A match concept B?" with verifiable certainty, the system needs:

1. A finite grammar for concept structure
2. A shared ground truth (bedrock) for term definitions
3. A recursive decomposition engine that can compare subcomponents
4. A deterministic stopping rule
5. A verifiable record of every comparison made

Each of these requirements will force a specific component in the sections that follow.

2.5 The UNSPSC Crisis (Motivating Case)

The United Nations Standard Products and Services Code (UNSPSC) was established to provide a universal taxonomy for classifying products and services across all procurement categories. In practice, it has fractured. An "antiviral pharmaceutical preparation" may be coded under multiple UNSPSC branches depending on whether the manufacturer, the logistics provider, or the regulatory body is assigning the code. A medical bone anchor and an aerospace bolt may sit in radically different UNSPSC segments despite

sharing structural properties. The cost of this fracture runs into the billions annually in misordered parts, delayed shipments, and manual reconciliation.

2.6 Three Alignment Problems — Where SOAM Sits

There are three distinct alignment problems, and it is essential to distinguish them:

Problem	Question	What SOAM Does
Semantic alignment	Does this symbol mean the same thing to both systems?	Solves this. Makes symbols verifiably equivalent across lexicons.
Goal specification	Have we written down the right objective?	Enables this. Precise semantics are a prerequisite for precise specification — but SOAM does not tell you <i>what</i> goals to set.
Value learning	Does the system infer what humans actually care about?	Enables this. Provenance and traceability are prerequisites for accountable inference — but SOAM does not provide the learning algorithms.

Semantic alignment — what SOAM solves — is **necessary but not sufficient** for goal specification and value learning. You cannot specify goals precisely or learn values accurately if the symbols you are using are ambiguous.

3. Related Work

Numerous approaches have attempted to solve the concept alignment problem, though none fully delivers determinism with auditable traceability.

Approach	Key Limitation
RDF/OWL reasoners	Open-world assumption prevents closed-world verification; predicate explosion
S-Match, Falcon-AO, LogMap	Probabilistic matching, no verifiable proof trace
AML (AgreementMakerLight)	Strong UI but no formal grounding to bedrock
BERTMap	LLM-based matching; hallucination risk; no determinism
Vector-only RAG	No structural verification, no unit conversion, no provenance
Human expert panels	Does not scale; expensive; inconsistent

The RDP diverges from all of these by combining: a fixed 18-slot grammar derived from SGF's orthogonal axes, recursive bisatisfiability checking with nested logical operators, metric-aware constraint matching with unit conversion, Prime Registry termination, and verifiable ProofTraces — all backed by a persistent adjacency manifest for deterministic edge execution.

4. Design Principles and Axioms

The RDP is founded on five design principles and six axioms. These are not claimed as universal truths — they are architectural constraints chosen because they produce the desired properties of determinism, auditability, and groundedness.

4.1 Five Design Principles

I. The Mandate of Reality: A concept exists within the system if its functional components can be traced to an immutable bedrock (the Prime Registry). Ungrounded concepts are identified and excluded from high-consequence alignment.

II. The Shift to Determinism: The system replaces statistical probability with structural logic, achieving deterministic comparison rather than probabilistic scoring.

III. The Transition of Utility: The objective is to specify the requirements for a thing's function, not to convey the "idea" of a thing.

IV. The Integrity of Recursion: Complexity is a dual-axis structure.

Generations define taxonomic lineage via the `IS_A` channel (Y-axis). **Plies** define functional composition via `HAS_PART` and Operational Channels (X-axis).

V. The Law of Taxonomic Determinism: An entity must possess a unique functional lineage for any given context. Polyhierarchy (multiple `IS_A` parents) is permitted but context disambiguates which parent is operationally relevant.

4.2 Six Axioms

Axiom I (Operational Reality): A concept exists within the system if its functional components trace to an immutable Bedrock.

Axiom II (Deterministic Cognition): By replacing statistical probability with structural logic, the system achieves deterministic comparison.

Axiom III (Transition of Utility): The objective is to specify requirements for function, not to convey the "idea" of a thing.

Axiom IV (Recursive Integrity): Complexity is dual-axis: **Generations** define taxonomic lineage; **Plies** define functional composition.

Axiom V (Taxonomic Determinism): An entity must possess a unique functional lineage for any given context. Polyhierarchy is permitted; context disambiguates.

Axiom VI (Structural Unambiguity): A concept is not a string; it is an address. Semantic disambiguation occurs at the point of storage, not access.

5. The Schema: A Problem-Derived Framework for Concept Definition

5.1 The Problem That Forces a Schema

Embedding similarity solves the throughput problem — it is fast, it is multilingual, it is good enough for search. But it does not solve the grounding problem. Two terms can have similar embeddings but refer to different concepts. "Screwdriver" the tool and "screwdriver" the cocktail are close in embedding space; so are "socket head cap screw" and "set screw." The reader who trusts only the embedding will be misled.

This forces a structured schema for concept definition. The schema must capture what a concept is, what it does, what it is made of, and what constraints it carries. Without this structure, the system can rank candidates but cannot verify equivalence. The question becomes: what is the minimal set of slots that captures all relevant structural information?

5.2 The Y-Axis: Structural Anchors (Object Logic)

The Y-axis defines what a concept *is* and what it is *made of*. Without these slots, the system cannot distinguish a "wagon" from a "cart" from a "carriage" — they may share embeddings and keywords, but their taxonomic lineage and component structures differ.

#	Slot	Type	Definition

1	IS_A	Set[Concept]	The taxonomic parent(s). Polyhierarchy permitted — a concept may have multiple IS_A parents (e.g., <code>en.tomato.fruit.noun</code> IS_A <code>en.fruit.food.noun</code> AND <code>en.vegetable.food.noun</code>).
2	HAS_PART	Set[Concept]	The functional components that compose this entity.

IS_A is necessary because without lineage, two concepts cannot be compared for type compatibility. HAS_PART is necessary because without composition, a complex concept cannot be decomposed into its functional elements.

5.3 The X-Axis: VerbHub and 15 Semantic Roles (Event Logic)

The Y-axis solves the "what is it" problem. But it does not solve the "what does it do" problem. A "screwdriver" and a "hammer" are both tools (same IS_A parent) and both have handles and metal ends (similar HAS_PART). The Y-axis alone cannot distinguish them.

This forces an X-axis. The X-axis captures the function or action associated with the concept — the verb that defines what it does, the participants that interact with it, and the circumstances under which it operates.

At the center is the **VerbHub**, holding the default action or function. Around it radiate the **15 SGF semantic roles**, matching SGF's closed role grammar exactly.

5.3.1 The VerbHub

The VerbHub stores the canonical verb ID and its features: actuality status (did it happen?), polarity (positive or negative), and modality (is it necessary, possible, or actual?).

The VerbHub also carries a rhetorical mode field that records the distance between the surface form of an utterance and its interpreted meaning. The mode is a typed value from a closed set: LITERAL (surface matches intent), SARCASTIC (surface conveys the opposite), HYPERBOLIC (exaggerated for effect, not factual), METAPHORICAL (figurative mapping to another domain), IRONIC (outcome contradicts expectation), and UNDERSTATED (deliberately minimized). A procurement officer who receives "I'd love to meet that spec" from a vendor cannot know from the words alone whether this is enthusiasm or sarcasm. The rhetorical mode field on the VerbHub captures this distinction. A high divergence score between surface form and interpreted meaning on a safety-critical utterance triggers a confirmation interrupt before acceptance. The Divergence Score is computed as the cosine distance between the Canonical Description embedding of the surface verb and the interpreted event.

5.3.2 The 15 SGF Semantic Roles

#	Slot	Type	Definition
3	HAS_AGENT	Concept	Entity that deliberately initiates the event
4	HAS_PATIENT	Concept	Entity that undergoes structural change
5	HAS_THEME	Concept	Entity that is moved, located, or possessed
6	HAS_EXPERIENCER	Concept	Living entity experiencing a psychological or sensory state
7	HAS_RECIPIENT	Concept	Destination entity that changes possession
8	HAS_BENEFICIARY	Concept	Entity for whose advantage the event occurs

9	HAS_TIME	Concept	Temporal coordinate, span, or constraint
10	HAS_LOCATION	Concept	Spatial region or coordinate
11	HAS_SOURCE	Concept	Origin state from which entity moves
12	HAS_DESTINATION	Concept	Endpoint state toward which entity moves
13	HAS_MANNER	Concept	Style, speed, or quality of execution
14	HAS_INSTRUMENT	Concept	Tool or intermediary force
15	HAS_CAUSE	Concept	Non-volitional trigger
16	HAS_REASON	Concept	Motivational purpose or mandate
17	HAS_ATTRIBUTE	Set[Concept]	Properties, constraint values, qualities

These match SGF's closed role set exactly.

5.4 The Constraint Slot

The Y-axis and X-axis solve the structural and functional problems. But they do not solve the quantitative problem. A "flight-qualified torque driver" and a "handheld screwdriver" may share structural and functional slots — both are tools, both drive fasteners. The difference is quantitative: torque accuracy, temperature range, material strength.

This forces a constraint slot for quantitative restrictions.

#	Slot	Type	Definition
18	HAS_CONSTRAINT	Set[Metric]	Quantitative restrictions

Each Metric specifies a property name, a numerical value with unit, a comparison operator (equals, greater-than, less-than, range), an optional tolerance, and a provenance source.

5.5 What the Schema Still Does Not Solve

The schema solves the grounding problem — it provides a fixed grammar for concept definition. But it creates a new problem. It does not specify how an empty slot should be treated. If one concept has a HAS_PART slot and another does not, should they be considered equivalent? Should the presence of any slot in one concept require its presence in the other?

*This forces a rule for empty slots: any slot may be empty — it imposes no matching requirement. However, if a slot is populated in **both** lexicons, it **must** match.*

5.6 What the Boolean Operators Solve

The schema with empty slots solves the partial-information problem. But it does not solve the conditional-requirement problem. A procurement specification might state: "the container must be made of aluminum OR stainless steel." A simple slot filler cannot express this — it would need multiple disjoint conditions.

This forces slot fillers to be logical expressions built from atomic references (concept IDs, metrics, or other slots) combined with AND, OR, XOR, and NOT operators. The default operator is AND — all conditions must be satisfied.

5.7 The Systemic Pointer

The logical expression system solves the conditional-requirement problem. But it does not solve the abstract-logic problem. When decomposition

encounters mathematical formulas, algorithmic constraints, or computational procedures, the recursion cannot proceed — there is nothing to decompose.

This forces a Systemic Pointer within the `HAS_INSTRUMENT` slot. The pointer halts semantic recursion and routes to an isolated execution sandbox — a read-only, memory-isolated environment with no filesystem or network access and a configurable timeout. The sandbox validates structural inputs and outputs against Metric tolerances without decomposing the abstract logic further.

6. The Rules of the Map

6.1 Foundational Rules (F)

F1 — Rule of Essence: Define by functional requirement, not physical implementation.

F2 — Rule of Recursive Integrity: Every node must be defined using the Schema.

F3 — Rule of Prime Registry Termination: Recursion must terminate at an NSM Prime or Physical Constant.

F4 — Rule of Exclusion: Use `NOT` only for conceptually adjacent logical traps.

F5 — Rule of Categorical Boundary: If decomposition hits Math/Logic, terminate with a Systemic Pointer.

F6 — Rule of Proximity (Polyhierarchy): Select the closest taxonomic parent for the given context. Polyhierarchy is permitted. For alignment, SOAM checks whether `IS_A` parent sets overlap: a partial overlap (A has parent X, B has parent X) is a pass, subject to the embedding similarity threshold. If no common parent exists, the axis check fails. No cycles.

F7 — Rule of Canonical Identity: Every node addressed by Canonical ID (`lemma.microgloss.pos`).

F8 — Rule of Downward Linkage: Every concept in L1–L4 must maintain explicit `IS_A` cross-layer edges to the Core Lexicon (L0). Floating Nodes are excluded from Tier 2+.

6.2 Optimization Rules (O)

O1 — Rule of Parsimony: Remove non-essential parts and roles.

O2 — Rule of Polymorphism: Define interfaces as parents, implementations as children.

O3 — Rule of Inheritance (The Difference Engine): A node stores only differential data not already established in its lineage. The manifest contains only the delta, not the full redefinition.

Incorrect: `calico_cat HAS_PART : [endoskeleton, tricolor_coat]` — endoskeleton is inherited from `mammal`.

Correct: `calico_cat HAS_PART : [tricolor_coat]`.

7. Quality Tests

7.1 N-Ply Stress Test: Decompose a concept to 3+ Plies. If the definition drifts or loses semantic intent along the way, **failed**.

7.2 Taxonomic Drift Test: If the `IS_A` parent is more than 1 Generation removed (without polyhierarchy justification), **failed**.

7.3 Prime Registry Verification: If terminal nodes are vague or non-primitive, **failed**.

7.4 Reversibility Test: Given the Canonical ID, can the concept be decomposed and resolved against the lexicon to recover the original sense?

If not, **failed**.

7.5 Traceability Audit: For any node in L1–L4, can every slot filler be traced to the Core Lexicon? If any filler is Floating, **failed** for Tier 2+.

7.6 Symmetry Test: Given two equivalent concepts defined by independent teams, does the alignment produce the same result regardless of which is treated as source and which as target? If not, **failed**.

7.7 Empty Slot Stability Test: If a slot is populated in one concept and empty in the other, does the system correctly treat it as a wildcard? If it falsely rejects, **failed**.

8. The Receiver Policy: Consequence Forces Admission Standards

8.1 The Problem That Forces the Receiver Policy

The schema with logical operators and empty slots solves the concept-definition problem. But it does not solve the consequence problem. A customer browsing a parts catalog and a procurement officer certifying a flight-critical component have very different tolerance for error. A 0.85 embedding similarity that is good enough for search is catastrophic for a billion-dollar screwdriver decision.

This forces a Receiver Policy — a configuration object that governs how strictly an incoming concept is admitted. The policy specifies, per deployment and per query, how deep to decompose, how high the similarity thresholds must be, and which slots are required. The policy is not a single value — it is a tiered system that matches the consequence of being wrong.

8.2 Policy Structure

The Receiver Policy is a configuration object that governs how strictly an incoming concept is admitted. It specifies:

- The **domain** (aerospace, medical, legal, etc.)
- The **maximum decomposition depth** for recursive matching
- **Per-depth embedding similarity thresholds** (e.g., 0.90 at depth 1, 0.95 at depth 2)
- Which **slots are required** to be populated
- The **metric tolerance** for numerical comparisons
- The **language** of the incoming concept
- Whether matching is **bidirectional** (both sides must satisfy each other)
- The minimum **rigor tier** for admission
- Whether **L0 traceability** is required
- Whether the **fingerprint pre-filter** is used and at what Hamming distance threshold

The thresholds (0.85, 0.90, 0.95, 0.98, 1.0) are initial suggested values derived from typical BGE M3 similarity distributions for semantically equivalent vs. non-equivalent concept pairs. They should be calibrated empirically for each deployment domain.

8.3 Rigor Profile Tiers

Tier	Label	Depth	Thresholds	Traceability	Use Cases
0	Candidate	0	[0.85]	None	Web search, initial filtering
1	Validated	1	[0.90]	Core recommended	Medical terminology, legal docs
2	Admitted	2	[0.90, 0.95]	Core mandatory	Aerospace, pharma, nuclear

3	Certified	3	[0.90, 0.95, 0.98]	Core + Prime verified	Life-critical, treaty verification
4	Absolute	∞	[0.90, 0.95, 0.98, 1.0]	Core + Prime + audited	Nuclear command, biometric identity

8.4 What the Receiver Policy Still Does Not Solve

The Receiver Policy solves the consequence problem by defining *how strictly* to match. But it does not solve the execution-speed problem. Running full structural alignment at Tier 3 on every query — including the low-stakes ones — would be needlessly slow for the browsing case and needlessly expensive for the catalog case.

This forces two execution paths: Fast Path and Verified Path.

8.5 Fast Path vs. Verified Path: Consequence-Sensitive Matching

Path	Method	Speed	Verifiability	Suitable for
Fast Path	Embedding similarity + cosine + cross-attention re-rank	Fast (milliseconds)	None (probabilistic)	Low-consequence queries: catalog browsing, initial triage, "where is my order?"
Verified Path	Full SOAM structural alignment (IS_A, HAS_PART, VerbHub, 15 roles,	Slower (seconds)	Deterministic, verifiable	High-consequence queries: purchase decisions, safety-critical

	constraints) with ProofTrace			parts, regulatory compliance
--	---------------------------------	--	--	---------------------------------

Fast Path is the default for low-stakes interactions. It retrieves the top-K candidates using embedding similarity, then re-ranks them with cross-attention. This is fast and cheap, but it cannot verify correctness — it can only rank likelihood.

Verified Path is invoked only when the Receiver Policy tier is ≥ 1 or when the query explicitly requests verification. It runs the full SOAM pipeline (Phases 0–3) and produces either a deterministic match (with ProofTrace) or a GapReport showing exactly why the match failed.

8.5.1 Same Engine Architecture

Crucially, both paths are served by the **same codebase** — the same ingestion pipeline, the same lexicon stack, the same persistent storage. The difference is purely in the execution path:

- Fast Path bypasses Phase 3 (SOAM) entirely, using only Phase 0 (fingerprint pre-filter) for candidate ranking.
- Verified Path executes all four phases, including recursive bisatisfiability and metric-aware constraint matching.

8.5.2 Example: Call Center Triage

Customer query: "I need a brake rotor for a 1997 Toyota Tacoma."
→ Fast Path: Returns top 3 candidates (embedding similarity).
If customer confirms, order proceeds at Tier 0 (no verification).

Customer query: "I need the exact rotor - VIN [VIN], OEM part number [VIN]"
→ Verified Path: Full SOAM alignment. Returns:
- MATCH: One verified SKU with ProofTrace.
- FAIL: GapReport showing which slot (e.g., HAS_PART diameter) mismatched.
Agent (or chatbot) displays GapReport to operator/customer for correction.

This architecture eliminates the guessing problem described in §2.2: the customer never has to guess, and the agent never has to hope. The system

either returns a verified answer or a structured reason it cannot.

8.6 What the Two-Path System Still Does Not Solve

The Fast Path/Verified Path distinction solves the consequence-speed tradeoff. But it does not solve the depth-precision problem. Even within the Verified Path, different queries need different decomposition depths.

Verifying a "wagon" at L2 might be sufficient for a catalog entry, but verifying a flight-critical torque driver at L2 is insufficient — the system needs to decompose each component until it reaches irreducible primes.

This forces a three-level structural depth hierarchy.

8.7 The Three-Level Structural Depth Hierarchy

Level	Method	Depth	Speed	Verifiability	Suitable for
L1	BGE-M3 embedding + cosine similarity + cross-encoder rerank + POS filter	Surface only (micro gloss text)	~1ms	None (probabilistic)	Search, discovery, candidate generation, low-stakes browsing
L2	Direct ontology comparison (IS-A, HAS-PART, HAS-PURPOSE, HAS-ATTRIBUTE)	One level	~10ms	Partial (slot-level match flags)	Most federation, everyday alignment, catalog lookup

	— one level deep)				
L3	Recursive ontology decomposition (each part decomposed and compared, continuing down to NSM primes or shared Core Lexicon entries)	Full depth	~100 ms–1s	Full (ProofTrace with recursive bisatisfiability)	Safety-critical, legal, medical, technical specifications, cross-lingual verification

8.7.1 Relationship to Consequence Paths

The three structural levels and the two consequence paths are orthogonal but compose naturally:

Consequence Path	Default Structural Level	Rationale
Fast Path (Tier 0)	L1	Fast triage; no verification needed
Verified Path (Tier 1)	L2	Moderate confidence; one-level ontology check
Verified Path (Tier 2)	L2 or L3	Domain-dependent; L3 for safety-critical slots

Verified Path (Tier 3+)	L3	Full recursive decomposition required
-------------------------	----	---------------------------------------

8.7.2 Level 1 — Embedding Similarity

Method:

Each canonical ID in the lexicon has a **Canonical Description** — a structured text string combining lemma, microgloss, part of speech, gloss, synonyms, and example sentence. This description is embedded using **BGE-M3**, a multilingual embedding model that places English and German text in the same vector space. A query term is embedded and compared against all entries via **cosine similarity**, filtered by part of speech, and optionally re-ranked by a cross-encoder.

Example: `wagon` (English → German)

Query: `en.wagon.vehicle.noun` — Canonical Description: *"wagon. vehicle. noun. A four-wheeled vehicle for transporting goods, typically pulled by horses. Example: The farmer loaded hay onto the wagon."*

Top cosine matches in German lexicon:

German entry	Cosine	POS match
<code>de.Pferdewagen.Fahrzeug.Nomen</code>	0.87	Yes
<code>de.Waggon.Eisenbahn.Nomen</code>	0.83	Yes
<code>de.Karren.Handfahrzeug.Nomen</code>	0.79	Yes
<code>de.Anhänger.Zugfahrzeug.Nomen</code>	0.71	Yes

Result: Level 1 returns a ranked list of candidates. It is fast (~1ms) and good enough for search and discovery, but it cannot distinguish between a horse-drawn wagon (`Pferdewagen`), a railway car (`Waggon`), or a handcart (`Karren`) — they all look similar in embedding space.

When to use Level 1:

- Everyday lookup and browsing
- Candidate generation for higher levels
- Low-stakes queries where near-synonyms are acceptable
- Initial triage in call center chatbots (§11.5)

8.7.3 Level 2 — Direct Ontology Comparison

Method:

Each entry in the lexicon carries ontological structure one level deep: its immediate parent concepts (IS-A), its immediate component parts (HAS-PART), its primary function (HAS-PURPOSE), and its salient properties (HAS-ATTRIBUTE). Level 2 takes the top candidates from Level 1 and compares these structures directly.

Example: wagon vs. German candidates

English `en.wagon.vehicle.noun` **ontology (one level):**

IS-A: vehicle, conveyance
HAS-PART: container, platform, axels, wheels, hitch
HAS-PURPOSE: transport goods by being pulled by an animal
HAS-ATTRIBUTE: horse-drawn, four-wheeled

Candidate 1: `de.Pferdewagen.Fahrzeug.Nomen`

IS-A: Fahrzeug, Transportmittel
HAS-PART: Behälter, Plattform, Achsen, Räder, Deichsel
HAS-PURPOSE: Güter transportieren, von Pferd gezogen
HAS-ATTRIBUTE: pferdegezogen, vierrädrig

Property	Match?
IS-A	Yes — vehicle, conveyance all match

HAS-PART	Yes — all 5 parts match (container, platform, axels, wheels, hitch)
HAS-PURPOSE	Yes — transport goods by horse
HAS-ATTRIBUTE	Yes — horse-drawn, four-wheeled

Score: 4/4 — Strong match

Candidate 2: `de.Waggon.Eisenbahn.Nomen`

IS-A: Eisenbahnfahrzeug, Schienenfahrzeug
HAS-PART: Sitzreihen, Fenster, Türen, Kupplung
HAS-PURPOSE: Personen transportieren
HAS-ATTRIBUTE: schienengebunden, mehrsitzig

Property	Match?
IS-A	Partial — both are vehicles, but German is specifically rail vehicle
HAS-PART	No — seats, windows, doors, coupler vs. container, platform, axels, wheels, hitch
HAS-PURPOSE	No — transport people vs. transport goods
HAS-ATTRIBUTE	No — rail-bound vs. horse-drawn

Score: 0.5/4 — Weak match, reject

Candidate 3: `de.Karren.Handfahrzeug.Nomen`

IS-A: Handfahrzeug, Transportmittel
HAS-PART: Ladefläche, Griff, Räder

HAS-PURPOSE: Güter von Hand bewegen
HAS-ATTRIBUTE: einachsig, handgezogen

Property	Match?
IS-A	Partial — both are transport tools, but German is specifically hand-pushed
HAS-PART	Partial — wheels match, but container/platform/hitch are missing; has handle instead
HAS-PURPOSE	Partial — both transport goods, but German is by hand, not horse
HAS-ATTRIBUTE	Partial — single-axle, hand-pulled vs. four-wheeled, horse-drawn

Score: 1.5/4 — Partial match, near-synonym at best

Level 2 Result:

Candidate	Score	Verdict
de.Pferdewagen	4/4	Accept — same concept
de.Waggon	0.5/4	Reject — different concept (railway car)
de.Karren	1.5/4	Flag — near-synonym, different vehicle type

When to use Level 2:

- Most federation and alignment tasks
- Confirming that a candidate from Level 1 is structurally equivalent
- Everyday translation where precision matters more than speed
- Catalog lookup where the cost of a wrong match is moderate

8.7.4 Level 3 — Recursive Ontology Decomposition

Method:

Level 2 compares one level deep. Level 3 goes **all the way down** — it recursively decomposes each part of each entry and compares the sub-parts, continuing until both sides hit either:

1. **NSM primes** (Appendix B) — the 65 irreducible concepts (SOMEONE, SOMETHING, DO, HAPPEN, MOVE, ROUND, TOUCH, etc.)
2. **Shared Core Lexicon entries** — byte-identical canonical IDs that both sides already agree on

This is the same recursive bisatisfiability mechanism described in §10.7, but applied to the *structural decomposition* of the concept itself rather than to slot-level logical expressions.

Example: wagon vs. Pferdewagen — deep comparison of hitch

Level 2 confirmed that both wagon and Pferdewagen have a hitch / Deichsel. Level 3 decomposes the hitch itself.

English hitch ontology (recursive):

1. IS-A: coupling_device, connector
2. HAS-PART:
 - tongue → IS-A: projecting_bar → PRIME: LONG, THIN
 - pin → IS-A: fastener → PRIME: LONG, THIN, HOLD
 - loop → IS-A: ring → PRIME: ROUND, HOLLOW
3. HAS-PURPOSE: attach wagon to animal → PRIME: CONNECT, MOVE
4. HAS-ATTRIBUTE:
 - articulating → IS-A: jointed → PRIME: MOVE, BEND
 - detachable → IS-A: removable → PRIME: SEPARATE

German Deichsel ontology (recursive):

1. IS-A: Kupplungsvorrichtung, Verbinder
2. HAS-PART:

- Deichselstange → IS-A: Stange → PRIME: LONG, THIN, RIGID
- Beschläge → IS-A: Metallverbinder → PRIME: HOLD, METAL
- Kette → IS-A: Kettenglieder → PRIME: LONG, FLEXIBLE, CONNECT

3. HAS-PURPOSE: Wagen an Zugtier befestigen → PRIME: CONNECT, MOVE

4. HAS-ATTRIBUTE:

- starr → IS-A: unbeweglich → PRIME: NOT MOVE, RIGID
- gelenkig → IS-A: beweglich → PRIME: MOVE, BEND

Comparison:

Property	English hitch	German Deichsel	Match?
IS-A	coupling_device, connector	Kupplungsvorrichtung, Verbinder	Yes
HAS-PART	tongue, pin, loop	Deichselstange, Beschläge, Kette	Partial — English has a tongue+pin mechanism; German has a rigid pole+fittings+chain
HAS-PURPOSE	attach wagon to animal	Wagen an Zugtier befestigen	Yes
HAS-ATTRIBUTE	articulating, detachable	starr (rigid), gelenkig (articulating)	Partial — English is primarily articulating; German can be rigid or articulating

Finding: The English `hitch` and German `Deichsel` are **not structurally identical**. The English hitch uses a tongue-and-pin mechanism that

articulates freely. The German Deichsel is traditionally a rigid pole with chain fittings, though modern versions may articulate.

Level 3 Result:

```
SOAM_L3(wagon_en, Pferdewagen_de) =  
  IS-A: 1.0 (perfect match)  
  HAS-PART: 0.85 (4 of 5 parts match perfectly; hitch has structural  
  HAS-PURPOSE: 1.0 (perfect match)  
  HAS-ATTRIBUTE: 0.90 (horse-drawn, four-wheeled match; hitch articu  
  
  Overall: 0.94 – Strong match with documented structural difference
```

When to use Level 3:

- **Safety-critical systems:** surgical robots, autonomous vehicles, industrial equipment
- **Legal contracts:** where a term's precise structural meaning determines obligations
- **Medical terminology:** where a drug's mechanism of action must match exactly
- **Technical specifications:** where parts must be interchangeable
- **Regulatory compliance:** where the law defines a term by its components
- **Cross-lingual verification:** where the cost of a false match is catastrophic (the NASA screwdriver problem)

8.7.5 Why There Is No Level 4

Level 3 already reaches the **floor** — NSM primes and shared Core Lexicon entries. There is nothing below primes to decompose. `ROUND` cannot be broken into smaller parts. `MOVE` cannot be decomposed further. The recursion terminates naturally at the bedrock of meaning.

Level 4 would be:

- **Infinite regress** — asking "what is `ROUND` made of?" forever

- **Circular definitions** — defining ROUND in terms of CIRCLE, and CIRCLE in terms of ROUND
- **Philosophical, not engineering** — metaphysics, not knowledge representation

The three levels are complete. L3 already touches the floor.

8.7.6 The Pipeline in Practice

Query: "Find German equivalent of English 'wagon' for a technical manual"

Step 1 (L1): Embed → cosine search → candidates

1. `de.Pferdewagen.Fahrzeug.Nomen` (0.87)
2. `de.Waggon.Eisenbahn.Nomen` (0.83)
3. `de.Karren.Handfahrzeug.Nomen` (0.79)

Step 2 (L2): Compare direct ontology of top candidates

- `Pferdewagen` : 4/4 match → ACCEPT as primary candidate
- `Waggon` : 0.5/4 → REJECT
- `Karren` : 1.5/4 → REJECT

Step 3 (L3): Recursively decompose `Pferdewagen` for deep verification

1. `container` ↔ `Behälter` → MATCH (both IS-A enclosure, HAS-PART walls/floor/opening)
2. `platform` ↔ `Plattform` → MATCH (both IS-A flat_surface)
3. `axels` ↔ `Achse` → MATCH (both IS-A rod, HAS-PART spindle/bearings)
4. `wheels` ↔ `Rad` → MATCH (both IS-A circular, HAS-PART rim/spokes/hub/tire)
5. `hitch` ↔ `Deichsel` → PARTIAL (English articulating tongue+pin vs. German rigid pole+chain)

Result: `Pferdewagen` is the correct match (SOAM 0.94). Documented structural difference: hitch mechanism differs between English and German

designs. Flagged for human review if the manual describes hitch replacement procedures.

8.7.7 Cross-Lingual Property

Because all three levels operate on canonical IDs and their ontological structure — not on surface strings — the same algorithm works for cross-lingual matching without modification. The embedding model (BGE-M3) places English and German descriptions in the same vector space for L1. The ontological slots (IS-A, HAS-PART, etc.) are language-agnostic by construction for L2 and L3.

Implication for the NASA Screwdriver Problem (§2.2): A procurement officer searching for a "flight-qualified torque driver" in an English specification can match against a German vendor's "flugtauglicher Drehmomentschrauber" using the same three-level pipeline. L1 finds the candidate. L2 confirms the structural slots match. L3 verifies that the torque mechanism, calibration procedure, and environmental ratings are recursively equivalent. The system returns a verified match with ProofTrace — or a GapReport explaining exactly which sub-component failed.

8.7.8 Emergent Property: The Verifiable Interlingua

The three-level hierarchy, combined with the Synapse grammar's language-neutral proposition format (verb at hub, 15 fixed semantic roles as spokes), produces a machine interlingua. Vocabulary alignment (SOAM on concept definitions) and grammar alignment (SOAM on Synapse graphs) use the same engine, the same lexicon stack, the same ProofTrace format, and the same consequence-sensitive depth policy. The result is a deterministic translation system where every word choice is verifiable, every proposition is auditable, and the depth of verification is proportional to the cost of being wrong.

This is distinguished from LLM-based translation by its structural determinism: the system does not guess — it verifies. When it cannot verify, it explains why via a GapReport that pinpoints exactly which slot or sub-

component failed. The same three-level rigor (L1 for fast candidate generation, L2 for moderate confidence, L3 for full recursive verification) applies to propositions as to concepts — a policy decision driven by the consequence of being wrong.

The interlingua is not a separate component. It is an emergent property of combining SOAM's deterministic alignment engine with the Synapse grammar's fixed, closed role set. The system compiles meaning into a language-neutral layer — first at the concept level (vocabulary alignment), then at the proposition level (grammar alignment) — producing a verifiable bridge between any two languages. The reader is referred to Section 14.1 for a discussion of grammar alignment as future work.

This interlingua is transported between systems via the Hub Fact Format (HFF) and Act and Federation Protocol (AFP) — a wire protocol for grounded meaning analogous to TCP/IP for bytes and HTTP for documents. HFF carries Synapses with their Canonical IDs, provenance, and optional micro-lexicon segments that teach unfamiliar terms on first contact. AFP declares the sender's intent (INFORM, QUERY, REQUEST, COMMAND, etc.), enabling receiver-sovereign coordination between machines that have never met. The full HFF/AFP specification is published at <https://github.com/SymbolGroundingFramework/SGF-manifest>.

9. Bootstrapping and Semantic Alignment

9.1 The Layered Lexicon Stack

Layer	Name	Scope	Source	Size
L0	Core Lexicon	All natural language	Wiktionary dump	~1.7M entries
L1	Domain-Specific Lexicon	Industry-wide terms	ISO, MIL-STD, STANAG, INN	~10K–100K

L2	In-House Lexicon	Organization-specific	Internal glossaries	~1K–50K
L3	Corpus-Wide Lexicon	Document corpus	TF-IDF extraction	Variable
L4	Document-Specific Lexicon	Single document	Spec/RFP definitions	~100–1K

[v3.2] Wings / Polyglot Lexicon Language:

Each concept in the lexicon can be thought of as having **wings** — a set of equivalent definitions across languages, linked by the same Canonical ID. The Core Lexicon (L0) is a **polyglot resource**: a single concept ID like `en.wagon.vehicle.noun` resolves to equivalent entries in German (`de.Pferdewagen.Fahrzeug.Nomen`), French (`fr.chariot.vehicule.nom`), and any language present in the Wiktionary snapshot. These wings are not translations in the traditional sense — they are independently grounded definitions that share a fingerprint neighborhood. The system aligns them at the concept level, not the string level.

This polyglot architecture means a procurement officer querying in English and a vendor describing a part in German are not translating between languages — they are resolving to the same concept ID through their respective wings. The system compiles meaning, not text. The fingerprint pre-filter (§10.4) confirms the match in microseconds.

9.1.1 Why Wiktionary as Core Basis

Criterion	WordNet	Wiktionary
Entries	~200K	~1.7M
Languages	English only	100+
Update frequency	~3–5 years	Continuous

Technical coverage	Poor	Good
Cross-lingual	None	Native
License	Princeton-specific	CC-BY-SA / GFDL

9.2 Layer Precedence

When resolving a concept, the system searches from the most specific layer (L4) to the most general (L0), returning the first match found. This ensures that document-specific definitions take priority over domain standards, which take priority over the general Core Lexicon.

9.3 Bootstrapping from the Core Lexicon

The bootstrapping process proceeds as follows:

1. **Ingest:** A snapshot of Wiktionary is taken at a specific date.
2. **Sense extraction:** Individual senses are extracted and assigned provisional Canonical IDs with microglosses.
3. **Canonical Description construction:** Each sense is compiled into a structured description combining lemma, microgloss, part of speech, gloss, hypernyms, synonyms, and examples.
4. **Embedding generation:** BGE M3 embeddings and Content Fingerprints are computed under the Exact Profile Contract (Appendix C).
5. **IS_A DAG construction:** Embedding similarity proposes candidate parent concepts; symbolic rules (hypernym extraction, cycle detection, polyhierarchy support) determine final parentage.
6. **Prime Registry grounding:** Each entry is verified to have a finite IS_A path to the Prime Registry. Unresolved entries are marked as `UNRESOLVED`.
7. **Content hash computation:** A SHA-256 hash of the canonical bytes provides integrity verification.
8. **Release packaging:** The complete lexicon is assembled into a signed, versioned release manifest.

9.4 The Linkage Mandate

Every concept in L1–L4 must maintain explicit `IS_A` cross-layer edges to the Core Lexicon (L0). A node is **Floating** if any of its slot fillers cannot be traced to L0 through a chain of `IS_A` edges. Floating nodes are excluded from Tier 2+ alignment.

9.4.1 The Traceability Chain

L4: `screwdriver (Appendix C)`

L3: `screwdriver (NASA-issued, titanium)`

L2: `en.flight_qualified_torque_driver.tool.noun`

L1: `en.torque_limited_fastener_driver.tool.noun`

L0: `en.torque.physical_quantity.noun` ,
`en.fastener.object_that_joins.noun` , `en.driver.tool.noun` ,
`en.handle.grasping_element.noun`

Prime Registry: DO, MOVE, GOOD, TRUE, SOMETHING, BODY, etc.

9.5 The LLM-to-Lexicon Bridge

The LLM acts as a noisy translation layer. Its outputs pass through a strict pipeline that enforces separation between interpretation (the LLM's domain) and admission (the structural filter's domain):

1. **Vector Projection:** The LLM's candidate terms are projected into embedding space.
2. **Candidate Retrieval:** Top-K matches are retrieved via cosine similarity against the Core Lexicon.
3. **Cross-Attention Reranking:** Concept definition schemas are compared.
4. **Provenance Verification:** The `IS_A` traceability chain to Core is checked.
5. **Structural Filter:** Full SOAM alignment is executed.
6. **Final Admission:** Only if all checks pass.

The LLM proposes; the schema decides. This separation prevents the LLM's probabilistic guesses from entering the grounded database without verification.

10. The Structural Ontology Alignment Engine (SOAM)

10.1 The Problem That Forces an Alignment Engine

The Receiver Policy and three-level hierarchy solve the *rules of admission* — they define what thresholds to use and how deep to decompose. But they do not solve the *execution problem*. Given two concept definitions in the 18-slot schema, how does the system actually compare them? How does it handle recursive slot fillers, nested logical operators, metric conversions, and termination at primes?

This forces an alignment engine — a deterministic procedure that takes two concepts and produces a verifiable result. The engine operates in four phases, each solving a problem created by the phase before it.

10.2 The Four-Phase Pipeline

Phase	Name	What it does	Dependencies
0	Content Fingerprint Pre-Filter	Compute 86-char LSH; compare via Hamming distance	Embedding model, Exact Profile Contract
1	Parse	Convert raw text into SGF schema	GLEAN (upstream)
2	Normalize	Flatten ANDs, canonicalize OR branches, convert units	Core Lexicon

3	Match	Recursive bisatisfiability over slot fillers	Layered lexicon stack
---	-------	--	-----------------------

Phase 0/Phase 3 Governance

Phase 0	Phase 3	Result	Tier
PASS	Not run	ACCEPT (fingerprint only)	Tier 0 only
PASS	PASS	ACCEPT (full verification)	Tier 1+
PASS	FAIL	REJECT (structural mismatch)	Tier 1+
FAIL	Not run	FAST REJECT	All tiers

10.3 System Architecture

The system operates as a four-phase pipeline. The phases execute sequentially, with each phase passing its output to the next if the check succeeds.

Phase 0: Content Fingerprint Pre-Filter — Solving the Throughput Problem

Full SOAM alignment at Tier 3 is slow — seconds per comparison. Running it on every query, including the millions of casual browsing queries a call center handles daily, would be impractical. But the system cannot simply skip verification for all queries — it needs verification for the high-stakes ones.

This forces a fast pre-filter. The fingerprint pre-filter computes an 86-character Locality Sensitive Hash from the embedding of a concept's canonical description. Two concepts with similar meaning produce fingerprints with low Hamming distance. If the distance exceeds the policy threshold, the system fast-rejects without invoking the full engine. This handles the majority of comparisons in microseconds.

Process:

1. Compute 86-character Base64URL LSH from Canonical Description.
2. Compare via Hamming distance against target fingerprint.
3. If Hamming distance $\leq$ policy threshold $\rightarrow$ proceed to Phase 1.
4. If Hamming distance $>$ policy threshold $\rightarrow$ fast reject.

Phases 1–2: Parse and Normalize — Preparing for Comparison

The fingerprint pre-filter solves the throughput problem. But it creates a new problem: it works on surface-level descriptions, not on structured schema. A passed fingerprint tells the system that two concepts *might* be equivalent — but it does not tell the system what slots they have, what constraints they carry, or how they are decomposed.

This forces a parsing phase that converts raw text into the 18-slot schema, and a normalization phase that flattens logical expressions, converts units to base SI, and resolves synonyms across the lexicon stack.

Process:

1. Extract Microgloss and Canonical ID.
2. Flatten nested expression trees (AND/OR/XOR/NOT) into canonical form.
3. Convert all units to base SI.
4. Resolve synonyms across the layered lexicon stack (L4 through L0).
5. Verify that all slot fillers trace to the Core Lexicon.

Output: Normalized concept definitions ready for comparison.

Phase 3: SOAM Execution Engine — The Core Verification

The parse and normalize phases solve the preparation problem. But they create the central problem of the system: given two fully specified, normalized concept definitions, how does the system determine if they are equivalent? Answering this question requires a procedure that compares each slot, evaluates logical expressions, handles recursive decomposition, and terminates at primes — all while producing a verifiable record.

This forces the SOAM Execution Engine — the core of the system.

Process (six verification steps, executed in order):

1. **Y-Axis Verification:** Compare IS_A polyhierarchy (check for overlapping parent sets). Compare HAS_PART composition (check each part).
2. **X-Axis Verification:** Compare VerbHub (action/function). Compare all 15 populated SGF semantic roles.
3. **Metric-Aware Constraint Matching:** For each populated constraint, convert units to base SI, apply the comparison operator with tolerance.
4. **Recursive Bisatisfiability Depth Verification:** For any atomic concept reference that passes the embedding similarity threshold at the current depth, recursively decompose and compare its slot fillers.
5. **Logical Operator Evaluation:** Evaluate AND/OR/XOR/NOT expressions in slot fillers according to their truth tables.
6. **Prime Registry Termination Check:** Verify that all recursive branches have terminated at NSM primes or Physical Constants. If any branch has not terminated, mark as UNRESOLVED.

Output (all six steps pass): **Admitted Identity** with signed SGF ProofTrace, ready for HFF/AFP export.

Output (any step fails): **GapReport** documenting the exact break point, violated constraint, and actionable feedback for the sender.

10.4 Phase 0: Content Fingerprint Pre-Filter

10.4.1 Computation

The Content Fingerprint is computed by passing the Canonical Description through BGE M3 to produce a 1024-dimensional embedding vector. This vector is then projected against 516 random hyperplanes using a fixed, contract-defined seed. Each hyperplane yields a binary decision (0 or 1), producing a 516-bit string. This string is encoded in Base64URL (unpadded), yielding exactly 86 characters.

Why 516 bits? 516 bits provides approximately 2^{258} effective collision resistance accounting for LSH dimensionality — sufficient for billions of concepts. 516 divides evenly into 86 Base64URL characters ($516/6 = 86$). Fewer bits (e.g., 256) increase collision risk; more bits (e.g., 768) increase storage by 50% without proportional benefit for the typical concept volume.

10.4.2 The Alignment Rule

Right-truncation for precision scaling. Full 86-character (516-bit) for global collision resistance. Lower-precision deployments truncate from the right — never pad.

10.4.3 Integration with SOAM

The fingerprint pre-filter is the first gate in the alignment pipeline. If the Hamming distance between two fingerprints exceeds the policy threshold, the system returns a fast reject without invoking Phases 1–3. If the distance is within threshold, the system proceeds to full SOAM alignment. For Tier 0 queries, the fingerprint match alone is sufficient for acceptance.

10.4.4 Fingerprint vs. Cryptographic Hash

Function	<code>content_hash</code> (SHA-256)	<code>content_fingerprint</code> (LSH)
Purpose	Data integrity (tamper detection)	Semantic proximity (meaning detection)
Sensitivity	1-bit change → 50% bit flip	Small change → small bit flip
Role in RDP	Integrity field in Canonical ID	Pre-filter for fast candidate rejection
SGF rule	Proves bytes unchanged	Supports matching; does not prove identity

10.4.5 Cross-Lingual Property

Because fingerprints are computed from multilingual embeddings (BGE M3), equivalent concepts described in different languages project to nearly identical coordinates, producing fingerprints with low Hamming distance. Based on BGE M3's published cross-lingual alignment benchmarks, we project that semantically equivalent descriptions in English, French, German, and Spanish will produce fingerprints differing by 2–10 bits out of 516 — well within typical matching thresholds. Empirical validation on specific language pairs is planned future work.

Implication: The fingerprint pre-filter is language-agnostic. A French part catalog can be matched against an English specification at the database level without runtime translation. The system compiles meaning, not text.

Nuance: This property holds for languages well-represented in the embedding model's training data. Low-resource or highly idiomatic languages may exhibit greater drift. The fingerprint is a fast pre-filter, not a proof of equivalence.

10.4.6 Ghost Nodes

If a description fails to produce a stable fingerprint (high variance across passes) or cannot be traced to the Core Lexicon, the system mints a provisional **Ghost** node. Ghosts persist until additional metadata arrives (domain expert, subsequent document, or HFF packet containing the missing definition). Ghosts expire after a configurable TTL (default: 30 days).

10.4.7 The SAME_AS Cache

The SAME_AS cache stores candidate alignments suggested by fingerprint proximity. These are probabilistic, pre-filter level suggestions — they do not constitute verified equivalence. Entries may be promoted to the Bridge Map if confirmed by full SOAM alignment.

10.5 Phase 1–2: Parse and Normalize

Phase 1 converts raw text into the SGF schema by extracting the Canonical ID, microgloss, and slot fillers. Phase 2 normalizes the result: nested AND expressions are flattened, OR branches are canonicalized (sorted and deduplicated), units are converted to base SI, and synonyms are resolved across lexicon layers.

10.6 Phase 3: Schema Comparison and Recursive Bisatisfiability

10.6.1 Top-Level Entry

The alignment function takes two concept definitions, their respective lexicon stacks, a domain identifier, and a Receiver Policy. It iterates over all 18 schema slots, comparing each slot's expression in both concepts. If any slot comparison fails, the overall alignment fails and a GapReport is generated pinpointing the exact failure.

10.6.2 Bidirectional Slot Comparison

Every slot comparison is **bidirectional**: concept A must satisfy concept B's requirements, and concept B must satisfy concept A's requirements. If either direction fails, the slot fails. This prevents asymmetric alignments where one concept is a superset of the other.

10.6.3 IS_A Polyhierarchy Comparison

When comparing IS_A slots, the system checks whether the sets of parent concepts overlap. A partial overlap — where concept A has parent X and concept B also has parent X — is sufficient for a pass, provided the embedding similarity between the parents exceeds the threshold. If no common parent exists, the axis check fails.

10.6.4 Logical Operators and Expression Trees

Slot fillers are logical expressions that may combine atomic references with AND, OR, XOR, and NOT operators. The system evaluates these recursively:

- **AND**: All child expressions must match.

- **OR**: At least one child expression must match.
- **XOR**: Exactly one child expression must match.
- **NOT**: The inner expression must not match.

10.6.5 Example

A requirement might specify that a container must be made of aluminum OR stainless steel, AND must have either a tamper-evident seal with locking lid OR a child-resistant cap with screw top. The system evaluates this as a nested logical tree, not as a flat list of properties.

10.7 Metric-Aware Constraint Matching

When comparing quantitative constraints, the system first checks unit compatibility. If units differ (e.g., MPa vs. psi), it converts to the base unit before comparison. It then applies the specified operator (=, ≥, ≤, >, <, or range) with the specified tolerance. A metric passes if the offered value falls within the required range including tolerance.

10.7.1 Example: NASA Bolt Verification

Slot	Spec (NASA)	Offer (Contractor)	Match?
IS_A	en.titanium_alloy .material.noun	en.ti_6al_4v.all oy.noun	0.95
HAS_ATTRIBUTE (Tensile-Strength)	≥900 MPa	≥950 MPa	
HAS_ATTRIBUTE (Yield-Strength)	≥830 MPa	≥880 MPa	
HAS_ATTRIBUTE (Density)	4.43 ± 0.05 g/cm ³	4.42 g/cm ³	

HAS_ATTRIBUTE (Melting-Point)	$\geq 1668^{\circ}\text{C}$	$\geq 1650^{\circ}\text{C}$	✗
----------------------------------	-----------------------------	-----------------------------	---

10.8 Recursive Bisatisfiability

When comparing two atomic concept references, the system first checks if they are metrics (in which case it uses numerical comparison). If both are conceptual, it computes the cosine similarity between their embeddings. If the similarity exceeds the depth-appropriate threshold, it then checks whether to stop recursing:

- If either concept is a metric, stop.
- If either concept is a Prime Registry entry, stop — comparison is exact.
- If the current depth has reached the maximum configured depth, stop.
- If either concept has no further decomposition in the lexicon, stop.

If none of these stopping conditions apply, the system resolves both concepts in their respective lexicon stacks and recursively compares their slot fillers. This continues until all branches terminate at primes or the maximum depth.

10.9 Prime Registry Termination

The Prime Registry (Appendix B) contains approximately 65 NSM semantic primes and NIST Physical Constants. When recursion reaches a prime, comparison is exact — no further decomposition is attempted.

Why NSM primes? The Natural Semantic Metalanguage (Wierzbicka, Goddard) is the only established inventory of semantic primes that has been validated across more than 30 languages. Alternative grounding schemes — WordNet's root synsets (which are English-only and topologically inconsistent), SUMO's physical objects (which mix ontological and linguistic categories), or a custom set of mathematical primitives (which cannot handle semantic concepts like GOOD, WANT, or DO) — all fail the cross-lingual grounding requirement. NSM primes are not merely *a* bedrock — they are the

only bedrock that supports language-neutral termination for a system that must align concepts across English, German, French, Arabic, and any other language represented in the Core Lexicon.

Prime Registry as infinite loop prevention. The Prime Registry functions as a hardware-level return instruction for the recursive decomposition engine. When a decomposition chain reaches an NSM prime (e.g., DO , MOVE , GOOD , INSIDE), the recursion stack pops cleanly — no further decomposition is possible or needed. This prevents the semantic infinite loops that plague cyclic ontology systems, where definitions collapse into closed circles: *"a wagon is a cart; a cart is a vehicle; a vehicle is a transport; a transport is a wagon."* The Prime Registry guarantees termination in finite steps.

10.10 Multi-Layer Resolution and Provenance

When resolving a concept reference, the system searches the lexicon stack from most specific (L4) to most general (L0). Each resolution records which layer the definition was found in, enabling provenance tracking. For example, a term resolved from L1 (Aerospace domain lexicon) carries different authority than one resolved from L0 (Core Lexicon).

10.11 ProofTrace Generation

10.11.1 Structure

Every SOAM alignment produces a ProofTrace — a structured record of every comparison made. Each trace entry records:

- The **type** of comparison (surface or structural)
- The **result** (PASS, FAIL, or WILDCARD for empty slots)
- The **depth** in the recursion tree
- The **slot** being compared
- The **similarity score** and **threshold** for embedding comparisons
- The **atoms** being compared and their **provenance layers**
- The **metric values** and whether they matched

- The **logical operator** (AND, OR, XOR, NOT) if applicable
- **Child traces** for recursive comparisons
- **Forward and reverse traces** for bidirectional checks
- The **failure reason** and **failure depth** if the alignment failed
- The **fingerprint match status** and **Hamming distance**

10.11.2 Example: Passing Alignment

SOAM ALIGNMENT RESULT: PASS (Depth 2, Tier: Certified)

Phase 0 (Content Fingerprint):

✓ Hamming distance: 3 (threshold: 10) → PASS

Stack provenance:

A: L2 (NASA in-house) → en.flight_qualified_torque_driver.tool.noun

B: L2 (Contractor in-house) → en.torque_screwdriver.calibrated_drive

Y-Axis (Object Logic):

✓ IS_A: precision_fastener_tool ≈ calibrated_fastener_tool ($0.93 \geq 0$)

✓ Common parent at depth 2: fastener_tool (0.95)

✓ HAS_PART: handle ≈ handle (1.0), torque_mechanism ≈ torque_limiter

X-Axis (Event Logic):

✓ VerbHub: TURN ≈ ROTATE (0.91)

✓ HAS_INSTRUMENT: blade ≈ bit (0.89)

✓ HAS_REASON: fasten ≈ secure (0.94)

Constraint:

✓ accuracy: $\pm 2\% \approx \pm 1.5\%$ ($1.5 \leq 2$) [METRIC PASS]

Prime Registry check:

✓ All terminal nodes traced to primes

Concepts are EQUIVALENT (Certified).

10.11.3 Example: GapReport as Actionable Feedback

A GapReport is not a failure in the system sense — it is structured, actionable feedback. The sender can calculate a corrected next move (offer a closer parent, provide traceability information) rather than retrying blindly. This is the architectural difference between a 500 Internal Server Error

and a typed `REFUSE` with a reason. In government procurement specifically, this means a vendor's system receives explicit guidance on *how to fix the mapping* rather than an opaque rejection that requires a 3-month change order to resolve.

```
SOAM ALIGNMENT RESULT: FAIL (Depth 1, Tier: Admitted)

Phase 0 (Content Fingerprint):
  ✓ Hamming distance: 4 (threshold: 10) → PASS

Y-Axis:
  ✓ IS_A: precision_fastener_tool ≈ hand_tool (0.82)
    → FAIL: similarity 0.82 < threshold 0.90

Failure type: IS_A mismatch
Failure depth: 1
Violated constraint: embedding similarity (0.82 < 0.90)
[Sender action: IS_A parent of 'hand_tool' is too distant from
'precision_fastener_tool'. Offer a parent within 1 generation
or provide traceability to a common ancestor.]
```

10.12 Complexity and Resource Requirements

10.12.1 Runtime Complexity

Phase 0 (fingerprint) is linear in the embedding dimension (1024). Full SOAM at depth D is exponential in the worst case — each slot may contain multiple atomic fillers, each of which may recursively decompose into further slots. In practice, execution runs far below this bound due to early termination on first slot failure and the fact that most concepts have few populated slots.

10.12.2 Resource Requirements

Resource	Minimum	Recommended	Notes
RAM (Phase 0)	2 GB	8 GB	BGE M3 inference requires ~4 GB

RAM (Phases 1–3)	256 MB	1 GB	Parsing and comparison operations
Storage (Core Lexicon)	2 GB	8 GB	1.7M entries with embeddings
Storage (working DB)	500 MB	2 GB	Indexes, expression trees, cache
CPU	ARM Cortex-A72	x86_64, 4+ cores	Single-thread performance matters
Network	None required	None required	All computation local after ingestion
Human effort (domain mapping)	—	2–8 weeks	Mapping L1/L2 terms to Core Lexicon

Embedding model dependency: BGE M3 inference requires a GPU or NPU for real-time Phase 0 computation. For deployments without suitable hardware, fingerprints can be precomputed during ingestion and shipped with the database. Phases 1–3 require no GPU.

10.13 Storage Layer

All state is stored in a persistent adjacency manifest — a compact, indexed database that supports fast point lookups on local edge hardware. The manifest stores:

- The **node registry**: every canonical ID with its IS_A parents, constraint metrics, VerbHub reference, content hash, content fingerprint, and grounding status

- **IS_A edges:** normalized parent-child relationships for efficient querying
- **Operational channels:** slot fillers for each concept, keyed by slot name
- **Logical expression trees:** the structure of AND/OR/XOR/NOT expressions
- **SAME_AS cache:** candidate alignments from fingerprint proximity
- **Bridge Map:** verified alignments with full ProofTraces

The manifest is designed for local execution with no cloud dependencies. All alignment operations run entirely on the edge device after initial lexicon ingestion.

11. Use Cases

11.1 Mode 1: Ad-Hoc Spec Verification

Scenario: Government agency issues RFP. Contractor responds with product specification.

This use case is **inherently asymmetric**: the agency defines the requirements, the vendor proposes a solution, and the agency validates compliance. SOAM supports this asymmetry because the Receiver Policy governs which lexicon is authoritative — the agency's spec is the requirement, and the vendor's offer must satisfy it.

Process:

1. Phase 0: Compute fingerprint of RFP requirement. Compute fingerprint of contractor's spec. Compare Hamming distance. If within threshold, proceed.
2. Phase 1-2: Parse contractor's spec into SGF schema. Resolve terms across layered stack.
3. Phase 3: Run full SOAM at appropriate Receiver Policy tier.
4. Output: ProofTrace with PASS/FAIL and provenance information.

Counterparty burden: None. The contractor's product literature — which they already produce — is sufficient.

Vendor lock-in prevention: Because alignments are stored in the Bridge Map with their ProofTraces, the agency retains an auditable record of every concept mapping. When a contract transitions to a new vendor, the Bridge Map serves as the semantic ground truth — the incumbent's proprietary mappings do not need to be renegotiated from scratch. Competitive re-procurement becomes feasible because the semantic infrastructure is vendor-independent.

11.2 Mode 2: Lexicon Bridge Building

Scenario: Two organizations need ongoing lexicon alignment across thousands of procurement decisions.

Process:

1. Run SOAM on every concept pair of interest.
2. Store successful alignments in the **Bridge Map** (deterministic, verified). Store fingerprint-based candidate alignments in the **SAME_AS** cache (probabilistic, pre-filter level).
3. SAME_AS entries may be promoted to Bridge Map if confirmed by full SOAM. Bridge Map entries are authoritative.
4. Future alignments become O(1) lookups in the Bridge Map.

Bridge Map Format:

```
BridgeMap := {
  org_A: "NASA",
  org_B: "Contractor XYZ",
  alignments: [{
    concept_A: "en.flight_qualified_torque_driver.tool.noun",
    concept_B: "en.torque_screwdriver.calibrated_driver.noun",
    provenance_A: "L2 (NASA in-house)",
    provenance_B: "L2 (Contractor in-house)",
    equivalence_tier: 2,
    proof_trace: {...},
```

```
fingerprint_match: True,
hamming_distance: 3,
timestamp: "2026-06-22T14:30:00Z"
}]
}
```

11.3 Industrial Case Study: The UNSPSC Commodity Code Chasm

The United Nations Standard Products and Services Code (UNSPSC) was designed to provide a universal taxonomy for classifying products across all procurement categories. In practice, it fractured. The same product — a medical bone anchor — may be coded differently by manufacturer, logistics provider, and regulator. An aerospace bolt and a medical bone anchor may occupy different UNSPSC segments despite sharing structural properties.

Traditional Approach

A human procurement officer manually reviews each vendor's product against the spec. This does not scale.

RDP/SOAM Approach

Both concepts are parsed into the 18-slot SGF schema and compared via SOAM:

Slot	Medical Bone Anchor	Aerospace Bolt	Match?
IS_A	medical_implant	structural_fastener	✗
VerbHub	FIXATE	FASTEN	⚠
HAS_PATIENT	bone_tissue	metal_plate	✗
HAS_ATTRIBUTE (material)	biocompatible_titanium	high_strength_alloy	⚠

HAS_ATTRIBUTE (tensile)	—	≥ 900 MPa	✗
----------------------------	---	-----------	---

Result: FAIL. Despite being structurally similar objects (both are cylindrical, threaded, metallic), they are conceptually different: one anchors soft tissue to bone; the other secures structural components under load. The system correctly rejects the alignment.

11.4 Emergent Capability: Privacy-Preserving Cross-Border Verification

The architecture enables a scenario where a counterparty proves a component meets a specification **without exposing proprietary internal data**.

The Protocol

1. **Agency publishes** the spec as an RDP-decompressed schema, registering its Content Fingerprint and a required ProofTrace acceptance threshold.
2. **Contractor ingests internally** — GLEAN processes their proprietary blueprints into a local, temporary node registry. The system computes the Content Fingerprint locally. No data leaves the contractor's air gap.
3. **Contractor exports** only the 86-character Base64URL fingerprint and the resulting pass/fail metric flags via an HFF transport packet. No proprietary definitions, no internal structure — only the compressed semantic address and boolean results.
4. **Agency imports** the packet into their local engine. Because the Exact Profile Contract guarantees identical hyperplane projections, the agency can execute recursive bisatisfiability matching on the fingerprint and metric outputs.

5. **Result:** The agency verifies that the contractor's component satisfies the operational constraints **without ever seeing the contractor's internal data**.

Limitations

This is a **privacy-preserving verification protocol**, not a full zero-knowledge proof. The fingerprint reveals some semantic information — enough to cluster similar concepts. A party with access to a known set of fingerprints could potentially match a contractor's fingerprint to a specific component type. For scenarios requiring cryptographic privacy guarantees, the fingerprint can be combined with commitment schemes; this is future work.

Cross-Reference Note

This capability builds on the cross-lingual property described in Section 10.4.5, which enables language-agnostic fingerprint matching across multilingual descriptions.

11.5 Mode 3: Unified Call Center Engine

Scenario: A high-volume parts retailer, aerospace distributor, or government logistics center receives thousands of product identification queries daily through three channels — chatbot, interactive voice response (IVR), and human-assisted agents. Each channel must be able to handle both low-stakes browsing and high-stakes purchase decisions, using a single semantic infrastructure.

The RDP/SOAM stack serves as the **unified backend** for all three channels.

11.5.1 Channel Architecture

Each channel connects to the same engine, but selects the appropriate structural level based on the query's precision requirements (see §8.7):

Channel	Typical Query	Structural Level	Output

Chatbot	"Show me brake rotors for a 1997 Tacoma"	L1 (embedding)	Top-K candidates
Chatbot	"I need the exact rotor — VIN ends in 12345"	L2 or L3 (ontology)	Verified SKU or GapReport
IVR	"Say your part number"	L1 (if ambiguous) or L2 (if exact)	Confirm SKU or request re-entry
Human agent	Customer describes part verbally	L2 or L3 (verify)	ProofTrace + GapReport on agent screen

The crucial property: **the same codebase powers all three**. There is no separate "chatbot ontology" or "agent database." The same lexicon stack, same fingerprint pre-filter, same SOAM alignment engine, and same storage handle every interaction.

11.5.2 Agent Workflow with GapReport

When a human agent receives a customer call, the system can run Level 2 or Level 3 matching in real time. If the match fails, the agent sees a GapReport directly in their interface — not a generic error, but a structured, actionable message:

```
GapReport – Brake Rotor Identification
=====
IS_A mismatch at depth 1:
  Customer description: "slotted brake disc"
  SKU candidate:       "drilled brake rotor"
  Similarity: 0.78 (threshold: 0.90)
  Suggested action: "Offer a candidate with 'slot' as a HAS_ATTRIBUTE
                    or confirm that slots and holes are equivalent in this context."
```

The agent can read this to the customer, ask a clarifying question, and retry. This transforms the call from a guessing game into a collaborative verification process. The customer never has to guess which part is correct — the system either confirms it or tells both parties exactly what is missing.

11.5.3 Return to the Trillion-Dollar Problem

Section 2.2 described the cycle: a customer calls, the system returns three candidates, the customer guesses, and the wrong part ships. The unified call center engine breaks this cycle:

- **For low-stakes queries** (e.g., "where is my order?"), L1 is fast and sufficient — no verification needed.
- **For high-stakes queries** (e.g., "I need the exact brake rotor for a 1997 Tacoma with the 15-inch wheels and ABS"), L2 or L3 runs automatically. The customer does not guess. The agent does not guess. The system returns **one verified SKU with a ProofTrace** or a **GapReport explaining exactly why no existing SKU matches**.

The cost of a single wrong part in aerospace or defense can reach billions of dollars. The cost of misordered parts across all industries exceeds a trillion dollars annually. The unified call center engine provides the first infrastructure that can verify a product match before it ships — not just rank candidates and hope.

11.6 Mode 4: RFP-to-Proposal Response Verification — The Natural Culmination

11.6.1 The Problem That Forces Requirement Alignment

The previous use cases solve the single-concept alignment problem. A procurement officer can verify that a "flight-qualified torque driver" from one lexicon matches the same concept from another lexicon. But real procurement is not a single-concept problem. A 25-page RFP on [SAM.gov](https://sam.gov) contains hundreds of interlocking requirements — personnel qualifications, food safety standards, equipment specifications, delivery schedules,

compliance certifications — each with its own sub-requirements, constraints, and conditional logic. The vendor's proposal must satisfy all of them, with legal and financial consequences for every missed requirement.

This forces a requirement tree approach. The same engine that aligns a single concept must be applied hierarchically: first deconstruct the RFP into a tree of requirement concepts, then deconstruct the proposal into an offer tree, then align them node by node using SOAM.

The formal name for a vendor's submission is a **proposal** (or **RFP response**; in U.S. Federal Acquisition Regulation (FAR) terminology, it is a "proposal" under FAR Part 15 for negotiated procurements, or a "bid" under FAR Part 14 for sealed bidding).

11.6.2 The Requirement Tree

The first step is to **deconstruct the RFP into a hierarchical requirement tree** using the same LLM-to-Lexicon Bridge described in §9.5. An LLM bursts the RFP's natural language into a structured outline of requirement concepts, each defined using the 18-slot SGF schema. The result is a tree where:

- **Root node** = the RFP itself (IS_A: procurement_specification)
- **Child nodes** = each major section (e.g., "Personnel Requirements," "Food Safety Standards," "Equipment Specifications")
- **Leaf nodes** = individual requirements (e.g., "Kitchen staff must hold ServSafe certification," "Refrigeration units must maintain $\leq 40^{\circ}\text{F}$ ")

Each requirement node carries:

- **HAS_REASON** — the purpose or mandate behind the requirement (e.g., "ensure food safety")
- **HAS_CONSTRAINT** — quantitative limits (e.g., temperature, staffing ratios, delivery times)
- **HAS_ATTRIBUTE** — qualitative properties (e.g., "USDA organic," "locally sourced")

- **HAS_PART** — sub-requirements that must all be satisfied
- **Logical operators (AND/OR/XOR/NOT)** in slot fillers — for example, "the contractor must provide either on-site dining OR a delivery service, AND must meet USDA nutritional guidelines"

11.6.3 The Offer Tree

The same process is applied to the vendor's proposal. The LLM bursts the proposal into a parallel **offer tree** using the same schema, slots, and logical expression format.

11.6.4 Item-by-Item Comparison

SOAM is run on each requirement-offer pair:

1. **Phase 0:** Compute and compare fingerprints at the root level for fast rejection of non-conforming proposals.
2. **Phase 1–2:** Parse both trees into normalized schema.
3. **Phase 3:** For each node in the requirement tree, find the corresponding node in the offer tree and run SOAM.
 - Bidirectional check: does the offer satisfy the requirement? Does the requirement accept the offer?
 - Recursive decomposition: if a requirement has sub-requirements, each sub-requirement is compared recursively.
 - Logical operator evaluation: AND/OR/XOR/NOT in slot fillers are evaluated according to their truth tables.

11.6.5 Compliance Report

The output is a structured compliance report with three sections:

Section 1: Passed Requirements (with ProofTraces)

For every requirement the proposal satisfies, the system produces a ProofTrace showing exactly which slot comparison passed at which depth, including provenance and fingerprint match status.

Section 2: Failed Requirements (with GapReports)

For every requirement the proposal fails to satisfy, the system produces a GapReport pinpointing the exact failure — which slot, which sub-component, and which constraint was violated. Each GapReport includes actionable feedback: "Your proposal states kitchen staff have 'basic food safety training' but the RFP requires 'ServSafe certified.' The similarity between 'basic food safety training' and 'ServSafe certification' is 0.72 (threshold: 0.90)."

Section 3: Aggregate Summary

The system aggregates all results according to the RFP's own logical structure:

```
COMPLIANCE REPORT: RFP-2026-EMBASSY-FOOD-SERVICE
```

```
=====
```

```
Overall: PASS (with conditions)
```

```
Section 1 (Personnel): PASS (8/8 requirements met)
```

```
Section 2 (Food Safety): PASS (5/5 requirements met)
```

```
Section 3 (Equipment): CONDITIONAL (3/4 requirements met)
```

```
- FAIL: Refrigeration temperature (proposal: ≤45°F, RFP: ≤40°F)
```

```
  GapReport: Temperature constraint violated by 5°F
```

```
- Vendor may correct by providing refrigeration spec that meets ≤40°
```

```
Section 4 (Logistics): PASS (6/6 requirements met)
```

```
Section 5 (Compliance): PASS (4/4 requirements met)
```

```
Verdict: Proposal is substantially compliant. Two conditional gaps  
identified. Vendor has 14 days to remediate or provide justification.
```

11.6.6 Benefit Over LLM-Only Approaches

An LLM alone can read an RFP and a proposal and produce a human-readable summary. But it cannot:

- Produce a **verifiable ProofTrace** for each requirement.
- Guarantee that it did not **hallucinate** a match.

- Provide **actionable GapReports** with precise similarity scores and thresholds.
- Aggregate results according to the RFP's **logical structure** (AND/OR/XOR/NOT).
- Allow **audit** — a contracting officer can inspect the ProofTrace for any clause.

SOAM does all of these. The LLM is still used for the initial parsing step (bursting prose into schema), but the verification is structural, deterministic, and auditable.

11.6.7 This Use Case Is Not Separate — It Is the Natural Scale

Vocabulary alignment (Mode 1–3) and requirement alignment (Mode 4) are not different systems. They are the same engine applied at two scales. The same four-phase pipeline, the same 18-slot schema, the same three-level depth hierarchy, the same ProofTrace format, and the same consequence-sensitive Receiver Policy govern both. A single concept and a 25-page specification differ only in the number of nodes in the tree. The architecture is fractal: the same mechanism that aligns a screwdriver aligns an entire procurement.

12. The Semantic Firewall Router

An emergent capability arising from placing the RDP/SOAM stack between untrusted LLM outputs and critical infrastructure.

12.1 The Threat

LLMs acting as procurement agents, code generators, or autonomous planners may generate commands that *look* structurally valid but refer to the wrong grounded concept — a "socket head cap screw" where a "set screw" is required.

12.2 The Defense

Every LLM-generated command passes through the full alignment pipeline before execution:

1. Parse the command's concept references into the 18-slot schema.
2. Resolve each reference against the organization's SGF lexicon stack.
3. Run SOAM against the authorized concept registry.
4. Admit or refuse the command with a verifiable GapReport.

12.3 The Difference

An LLM can suggest plausible-looking concept references. The Semantic Firewall Router intercepts those references before they reach actuators, databases, or other systems, and subjects them to the same deterministic alignment that governs cross-organization procurement. The LLM proposes; the schema decides.

13. Limitations

Limitation	Severity	Mitigation
Fingerprint collisions (LSH)	Low	Resolved by Phase 3 SOAM — fingerprint is a pre-filter, not a verdict
Embedding non-determinism	Medium	Exact Profile Contract locks model version; cross-deployment determinism requires identical hardware and precision settings
Cold start cost	High	Initial lexicon construction is computationally expensive; mitigated by tiered adoption (start at Tier 0, upgrade as needed)

Low-resource language coverage	Medium	BGE M3 covers ~100 languages; low-resource languages need evaluation
Privacy leakage via fingerprint	Medium	Fingerprint reveals semantic neighborhood; cryptographic commitments are future work
Parser quality	High	Quality varies by domain and language; domain-specific parsers are planned
Schema completeness	Low	18-slot schema is extensible via domain profiles
Governance overhead	Low	Omega integration is specified but not yet implemented
Version skew (LO changes)	Medium	LexiconRelease manifest with version pinning; cross-system negotiation is future work
Requirement tree granularity	Medium	The granularity of the deconstructed requirement tree depends on LLM parser quality; a mis-split requirement may produce misleading results. Mitigation: human review of the parsed tree before running SOAM.
Semantic alignment ≠ value alignment	None (scope clarification)	This paper solves semantic alignment only; goal specification and value learning are separate problems that semantic alignment enables but does not replace

14. Conclusion and Future Work

14.1 The Derivation Chain

This paper began with a single problem: two high-fidelity technical lexicons cannot efficiently determine whether they refer to the same grounded concepts. From that problem, each component of the system was forced into existence:

1. Embedding similarity alone fails (Section 2.3) → **This forces a structured schema** (Section 5).
2. The schema provides slots but not matching rules → **This forces rules and quality tests** (Sections 6–7).
3. The rules work but different stakes need different rigor → **This forces the Receiver Policy with tiers** (Section 8).
4. The policy defines thresholds but not execution → **This forces the SOAM engine** (Section 10).
5. Full SOAM on every query is too slow → **This forces the fingerprint pre-filter** (Phase 0).
6. The pre-filter is fast but probabilistic → **This forces verified alignment with ProofTrace** (Phase 3).
7. Different verification depths are needed for different stakes → **This forces the three-level hierarchy** (L1–L3).
8. Vocabulary alignment works but real procurement involves complex specs → **This forces requirement trees** (Section 11.6).

Each component is not a design choice. Each component is the only possible answer to the problem created by the component before it. The architecture is a chain of necessity, not a tree of invention.

14.2 The Capabilities That Emerge

The architecture yields four capabilities that span from a single concept to a full procurement specification:

Capability	Section	Scope

Privacy-preserving verification	§11.4	A counterparty proves compliance by exporting only an 86-character fingerprint and pass/fail flags
Cross-lingual interoperability	§10.4.5	Language-agnostic database lookups via multilingual embeddings
Unified call center engine	§11.5	Single codebase for chatbot, IVR, and human agent channels
RFP-to-proposal response verification	§11.6	Full specification alignment with item-by-item ProofTraces

Language is vocabulary plus grammar. SOAM provides the vocabulary layer — deterministic concept alignment with ProofTraces. The Synapse format provides the grammar layer — a fixed-role proposition structure that is itself alignable by the same engine. Together they form a complete, verifiable interlingua: the vocabulary tells you what things mean; the grammar tells you who did what to whom. Both are auditable, both are consequence-sensitive, and both use the same deterministic architecture.

A fifth property emerges from the combination of SOAM vocabulary alignment and the Synapse grammar's language-neutral proposition format: **a verifiable interlingua** (Section 8.7.8). When vocabulary and grammar are both aligned under the same deterministic engine, the result is a complete machine translation system where every word choice carries a ProofTrace and the depth of verification is selected by the consequence of being wrong.

14.3 The Fractal Architecture

Vocabulary alignment, grammar alignment, and requirement alignment are not three systems. They are the same system applied at three scales — concept, proposition, and specification. Each uses the same four-phase pipeline, the same 18-slot schema, the same three-level depth hierarchy, the same ProofTrace format, and the same consequence-sensitive Receiver

Policy. The reader who understands how SOAM aligns a single screwdriver understands how it verifies a 25-page procurement specification. The architecture is fractal: the same mechanism at every magnification.

14.4 The Strongest Claim

Within the broader SGF ecosystem, the RDP produces grounded lexicon entries that feed into Synapse grammar, GLEAN ingestion, HFF transport, and Omega governance. It makes the strongest claim that is architecturally defensible: **semantic alignment infrastructure means that when you procure an AI system for government use, you can require that its terms resolve through a shared lexicon, that it produce GapReports rather than opaque failures, and that every decision carry a verifiable ProofTrace. These are auditable contractual requirements — not prompts you hope the model honors.**

14.5 Future Work: From Vocabulary to Propositions

SOAM addresses vocabulary alignment — determining whether a concept in one language or ontology is equivalent to a concept in another. It also addresses requirement alignment (Section 11.6) by applying the same engine to hierarchical requirement trees. It does not address grammar alignment — comparing the propositional structure of sentences across languages.

However, SGF's Synapse format provides a language-neutral representation of propositions, with a verb at the hub and 15 semantic roles as spokes. A natural extension of this work would apply SOAM's recursive bisatisfiability to Synapse graphs, enabling deterministic verification of full propositions across languages. This would constitute a complete interlingual machine translation system with verifiable ProofTraces — verifying not just that `wagon` matches `Pferdewagen`, but that "the horse pulls the wagon" matches "das Pferd zieht den Wagen." This capability would address both vocabulary and grammar, providing a deterministic alternative to LLM-based translation for high-consequence domains where verification is non-negotiable.

This extension is left for future work.

[v3.2] The companion specifications for the full SGF stack — including the Hub Fact Format (HFF) wire protocol, the Act and Federation Protocol (AFP), and the Omega governance language — are published at [repository URL]. These documents define how SOAM-aligned concepts and Synapse groups are transported between systems, how speech acts (INFORM, COMMAND, REQUEST, etc.) are typed and governed, and how permission policies are compiled into deterministic checks.

References

1. SGF Core Specification v1.0. Symbol Grounding Framework.
 2. Wierzbicka, A. (1996). *Semantics: Primes and Universals*. Oxford University Press.
 3. Goddard, C., & Wierzbicka, A. (2002). *Meaning and Universal Grammar*. John Benjamins.
 4. Goddard, C., & Wierzbicka, A. (2014). *Words and Meanings: Lexical Semantics Across Domains, Languages, and Cultures*. Oxford University Press.
 5. Fillmore, C. J. (1982). Frame Semantics. In *Linguistics in the Morning Calm*.
 6. Xiao, S., et al. (2024). BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embedding Through Contrastive Learning. *arXiv:2402.03216*.
 7. Euzenat, J., & Shvaiko, P. (2013). *Ontology Matching* (2nd ed.). Springer.
 8. Hipp, R. D. (2020). SQLite. <https://www.sqlite.org/>
 9. NIST. (2022). CODATA Internationally Recommended Values of the Fundamental Physical Constants.
 10. Wierzbicka, A. (1972). *Semantic Primitives*. Athenäum.
-

Appendix A: Slot Name Mapping (RDP to SGF Standard)

RDP v1.0 Name	SGF Standard Name	Axis
IS - A	IS_A	Y-axis
HAS - PART	HAS_PART	Y-axis
(new)	VerbHub (verb_canonical_id)	X-axis hub
AGENT	HAS_AGENT	X-axis
PATIENT	HAS_PATIENT	X-axis
THEME	HAS_THEME	X-axis
EXPERIENCER	HAS_EXPERIENCER	X-axis
RECIPIENT	HAS_RECIPIENT	X-axis
BENEFICIARY	HAS_BENEFICIARY	X-axis
INSTRUMENT	HAS_INSTRUMENT	X-axis
LOCATION	HAS_LOCATION	X-axis
SOURCE	HAS_SOURCE	X-axis
DESTINATION	HAS_DESTINATION	X-axis
TIME	HAS_TIME	X-axis
MANNER	HAS_MANNER	X-axis
CAUSE	HAS_CAUSE	X-axis
REASON	HAS_REASON	X-axis
ATTRIBUTE	HAS_ATTRIBUTE	X-axis
CONSTRAINT	HAS_CONSTRAINT	Constraint

Appendix B: NSM Prime Registry

Based on Goddard & Wierzbicka (2014). The exact inventory may vary by deployment; the Exact Profile Contract must specify which Prime Registry version is in use.

Category	Primes
Substantives	I, YOU, SOMEONE, PEOPLE, SOMETHING/THING, BODY
Relational substantives	KIND, PART
Determiners	THIS, THE SAME, OTHER/ELSE/ANOTHER
Quantifiers	ONE, TWO, SOME, ALL, MUCH/MANY, LITTLE/FEW
Evaluators	GOOD, BAD
Descriptors	BIG, SMALL
Mental predicates	THINK, KNOW, WANT, DON'T WANT, FEEL, SEE, HEAR
Speech	SAY, WORDS, TRUE
Actions, events, movement	DO, HAPPEN, MOVE
Existence and possession	BE (SOMEWHERE), THERE IS, BE (SOMEONE/SOMETHING), (IS) MINE
Life and death	LIVE, DIE
Time	WHEN/TIME, NOW, BEFORE, AFTER, A LONG TIME, A SHORT TIME, FOR SOME TIME, MOMENT

Space	WHERE/PLACE, HERE, ABOVE, BELOW, FAR, NEAR, SIDE, INSIDE, TOUCH (CONTACT)
Logical concepts	NOT, MAYBE, CAN, BECAUSE, IF
Intensifier, augmentor	VERY, MORE
Similarity	LIKE/AS/WAY

Physical constants (NIST CODATA 2022): c (speed of light), h (Planck constant), k_B (Boltzmann constant), e (elementary charge), π , ϕ (Euler's number), and others as defined by the Exact Profile Contract.

Appendix C: Exact Profile Contract Specification

The Exact Profile Contract is the set of parameters that must be identical across deployments for fingerprints and embedding comparisons to be interoperable.

Parameter	Value	Notes
Embedding model	BGE-M3	Specifically: BAAI/bge-m3 at revision v1.0
Dimensionality	1024	Output dimension of the embedding model
Pooling method	CLS	CLS token pooling
Normalization	L2	Unit vector normalization applied before hyperplane projection

Hyperplane count	516	Number of random hyperplanes
Hyperplane seed	0x5GF_RDP_2026	Fixed seed for reproducibility
Encoding	Base64URL (unpadded)	86 characters for 516 bits
Canonical Description format	lemma + " " + microgloss + " " + POS + ". " + gloss + ". Example: " + example	Structured text input to embedding model
Prime Registry version	Goddard & Wierzbicka 2014	Source for the 65-prime inventory
Physical constants version	NIST CODATA 2022	Source for physical constant values
Core Lexicon source	Wiktionary dump 2025-06-01	Specific snapshot date
Language coverage	en, fr, de, es, it, pt, nl, ru, zh, ja, ar +	Languages present in the Core Lexicon

Any deviation from this contract places the system in a distinct mathematical universe — fingerprints will not be comparable.

Appendix D: Terminology

Term	Definition
Bridge Map	Persistent store of verified (SOAM-confirmed) alignments between two lexicons. Authoritative.

Canonical ID	Structured address for a sense-level LexiconEntry. Format: <code>lemma.microgloss.pos</code> .
Content Fingerprint	86-character Base64URL LSH of a concept's Canonical Description. Supports candidate matching; does not prove identity.
Fast Path	Low-consequence execution path using only embedding similarity (bypasses Phase 3). Probabilistic, no verification.
Floating Node	A concept in L1–L4 that cannot be traced to the Core Lexicon (L0) via IS_A edges. Excluded from Tier 2+.
Ghost Node	Provisional node minted when a description cannot produce a stable fingerprint or be traced to Core. Expires after TTL.
GapReport	Truncated ProofTrace documenting the point of failure and the specific constraint or slot that caused it. Carries actionable feedback for the sender.
Level 1 (L1)	Embedding-only matching (cosine similarity + cross-attention re-rank) for fast candidate generation. Surface-level only.
Level 2 (L2)	Direct ontology comparison (IS-A, HAS-PART, HAS-PURPOSE, HAS-ATTRIBUTE) one level deep. Moderate confidence.
Level 3 (L3)	Recursive ontology decomposition of each part down to NSM primes or shared Core Lexicon entries. Full depth. Highest confidence.
Offer tree	The hierarchical structure of deconstructed requirements extracted from a vendor's proposal, formatted in the 18-slot SGF schema for comparison against the requirement tree.
Prime Registry	The set of ~65 NSM semantic primes and NIST Physical Constants at which all recursive decomposition terminates.

	Functions as a hardware-level return instruction.
ProofTrace	Verifiable record of every comparison made during a SOAM alignment, including slot, depth, similarity scores, and provenance.
Proposal (RFP response)	A vendor's formal submission in response to a Request for Proposal. Under U.S. FAR Part 15, this is a "proposal"; under FAR Part 14 it is a "bid." This paper uses "proposal" as the general term.
Receiver Policy	The governing logic dictating admission thresholds for incoming concepts: tier, depth, traceability requirements.
Requirement tree	The hierarchical structure of deconstructed requirements extracted from an RFP, formatted in the 18-slot SGF schema with logical operators (AND/OR/XOR/NOT).
SAME_AS	Cache of candidate alignments from fingerprint proximity (probabilistic, pre-filter level). May be promoted to Bridge Map after SOAM confirmation.
SAM.gov	The official U.S. federal government website for procurement opportunities, including RFPs for embassy, military base, and other government operations.
Semantic alignment	The problem of making sure two systems mean the same thing by the same symbols. What SOAM solves.
Shadow Map	A temporary, unverified concept mapping proposed by an LLM during ingestion. Must pass SOAM before admission.
Systemic Pointer	A reference in the <code>HAS_INSTRUMENT</code> slot that halts recursion and routes to an isolated execution sandbox for abstract logic.

Unified Call Center Engine	A single RDP/SOAM backend serving chatbot, IVR, and human agent channels, with seamless escalation from L1 to L3.
Verified Path	High-consequence execution path using full SOAM pipeline (Phases 0–3). Deterministic, produces ProofTrace.
Verifiabl e Interling ua	The emergent property of combining SOAM vocabulary alignment with Synapse grammar alignment — a deterministic translation system where every word choice carries a ProofTrace and the depth of verification is selected by the consequence of being wrong.
Y-Axis	Object logic axis: IS_A (type hierarchy) and HAS_PART (composition). What a thing <i>is</i> and <i>is made of</i> .
X-Axis	Event logic axis: VerbHub (action center) and 15 semantic roles (participants and context). What a thing <i>does</i> .

End of specification. Version 3.2 — June 2026.